

STUDY OF ARM EVALUATION SYSTEM**Objective:**

To study the ARM processor system and to describe the features of its architecture. Keil software

Software tools Requirement: Keil Software

Introduction to ARM7:

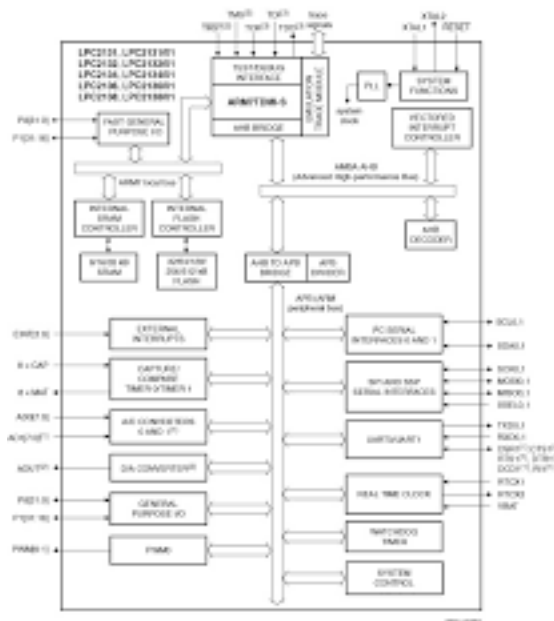
The LPC2141/42/44/46/48 microcontrollers are based on a 16-bit/32-bit ARM7TDMI-S CPU with real-time emulation and embedded trace support, which combine a microcontroller with embedded high-speed flash memory ranging from 32 kB to 512 kB. A 128-bit wide memory interface and unique accelerator architecture enable 32-bit code execution at the maximum clock rate. For critical code size applications, the alternative 16-bit Thumb mode reduces code by more than 30 % with a minimal performance penalty. Due to their tiny size and low power consumption, LPC2141/42/44/46/48 are ideal for applications where miniaturisation is a key requirement, such as access control and point-of-sale. Serial communications interfaces range from a USB 2.0 Full-speed device. multiple UARIS, SPI, SSP to 12C-bus and on-chip SRAM of 8 kB up to 40 kB, make these devices very well suited for communication gateways and protocol converters, soft modems, voice recognition, and low-end imaging, providing both large buffer size and high processing power. Various 32-bit timers. Single or dual 10-bit ADC(s), 10-bit DAC, PWM channels and 45 fast GPIO lines with up to nine edge or level-sensitive external interrupt pins make these microcontrollers suitable for industrial control and medical systems.

Features

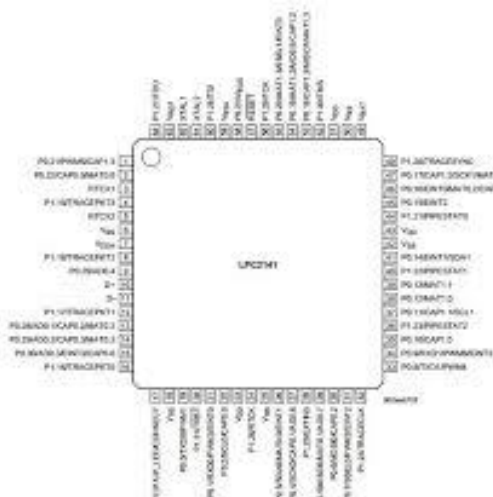
- 16-bit/32-bit ARM7TDMI-S microcontroller in a tiny LQFP64 package.
- 8 kB to 40 kB of on-chip static RAM and 32 kB to 512 kB of on-chip flash memory .
- 28-bit wide interface/accelerator enables high-speed 60 MHz operation.
- In-System Programming/In-Application Programming (ISP/IAP) via on-chip boot loader software. Single flash sector or full chip erase in 400 ms and programming of 256 bytes in 1 ms.
- EmbeddedICE RT and Embedded Trace interfaces offer real-time debugging with the on-chip RealMonitor software and high-speed tracing of instruction execution.
- USB 2.0 Full-speed compliant device controller with 2 kB of endpoint RAM. In addition, the LPC2 146/48 provides 8 kB of on-chip RAM accessible to USB by DMA.
- One or two (LPC2141/42 vs. LPC2144/46/48) 10-bit ADCs provide a total of 6/14 analog inputs, with conversion times as low as 2.44 ps per channel.
- Single 10-bit DAC provides variable analog output (LPC2142/44/46/48 only),.
- Two 32-bit timers/external event counters (with four capture and four compare channels each), PWM unit (six outputs) and watchdog.
- Low power Real-Time Clock (RTC) with independent power and 32 kHz clock input.
- Multiple serial interfaces including two LARTs (16C550), two Fast 12C-bus (400 kbit/s).

- SPI and SSP with buffering and variable data length capabilities,
- Vectored Interrupt Controller (VIC) with configurable priorities and vector addresses.
- Up to 45 of 5 V tolerant fast general purpose I/O pins in a tiny LQFP64 package.
- Up to 21 external interrupt pins available.
- 60 MHz maximum CPU clock available from programmable on-chip PLL with settling time of 100 us
- On-chip integrated oscillator operates with an external crystal from 1 MHz to 25 MHz.
- Power saving modes include Idle and Power-down.
- Individual enable/disable of peripheral functions as well as peripheral clock scaling for additional power optimization.
- Processor wake-up from Power-down mode via external interrupt or BOD

Architecture of ARM7 (LPC2148):



Pin Diagram:



Introduction to PSoC

When developing more complex projects, there is often a need for additional peripheral units, such as operational and instrument amplifiers, filters, timers, digital logic circuits, ADC and DA convertors, etc. As a general rule, the implementation of a printed circuit board increases power consumption...All of these factors can significantly affect the price of the extra peripherals, bringing in additional difficulties: new components take up space and require additional attention during the development cycle of the project.

The introduction of PSoC microcontrollers has made many engineers' dreams come true of having all their project needs covered in one chip.

PSoC: Programmable System on Chip:

PSoC (Programmable System on Chip) represents a whole new concept in microcontroller development. In addition to all the standard elements of 8-bit microcontrollers, PSoC chips feature digital and analogue programmable blocks, which themselves allow implementation of a large number of peripherals. Digital blocks consist of smaller programmable blocks that can be configured to allow different development options. Analogue amplifiers, as well as AD and DA converters. There are several different PSoC families you can base your project upon. Blocks are used for the development of analogue elements, such as analogue filters, comparators, and instrumental (non-) inverting, depending on the project requirements. The basic difference between PSoC families is the number of available programmable blocks and the number of input/output pins. The number of components that can be devised is primarily a function of the available programmable blocks.

Depending on the microcontroller family, PSoC chips have 4-16 digital blocks and 3-12 analogue programmable blocks.

Characteristics of PSoC microcontrollers

Some of the most prominent features of PSoC microcontrollers are:

- MAC unit, hardware 8x8 multiplication, with result stored in 32-bit accumulator,
- Changeable working voltage, 3.3V or 5V.
- Possibility of a small voltage supply, to 1V,
- Programmable frequency choice

Programmable blocks allow you to devise:

- 16K bytes of programmable memory
- 256 bytes of RAM
- AD convertors with maximum resolution of 14 bits
- DA convertors with maximum resolution of 9 bits.
- Programmable voltage amplifier,
- Programmable filters and comparators
- Timers and counters of 8, 16, and 32 bits
- Pseudorandom sequences and CRC code generators.
- Two Full-Duplex UARTs.

- Multiple SPI devices.
- Option for connection on all output pins.
- Option for block combining
- Option for programming only the specified memory regions and write protection
- For every pin, there is an option of Pull up, Pull down, High Z, Strong, or Open pin state
- Possibility of interrupt generation during change of state on any input/output pin
- I2C Slave or Master and Multi-Master up to speed of 400KH.
- Integrated Supervisory Circuit.
- Built-in precise voltage reference

System Overview:

The CPU unit is the main part of a microcontroller whose purpose is to execute program instructions and control the workflow of other blocks.

The frequency generator facilitates signals necessary for the CPU to work, as well as an array of frequencies that are used by programmable blocks. These signals could be based on an internal or external referent oscillator.

In the case of an irregular reset, the controller enables the microcontroller to start action and brings the microcontroller to regular state events.

Watch Dog timer is used to detect software dead-loops as a regular timer.

The sleep timer can periodically wake up the microcontroller from power-saving modes.

It could also be used as Input-Output pins enable communication between the CPU unit, digital and analogue programmable blocks, and the outside world.

Digital programmable blocks are used to configure digital programmable components, which are selected by user components, like AD and DA converters, filters, and DTMF.

Analogue configurable analogue programmable blocks are used to create operational amplifiers receivers, programmable, instrumental, inverting, and non-inverting.

An interrupt controller handles necessary operations in the case of interrupts.

The I2C controller enables hardware realisation of an I2C communication.

Voltage reference is vital for the work of analogue components that reside inside of analog programmable blocks.

The MAC unit is used for operations of hardware signed multiplication of 8-bit numbers. part of a voltage regulator.

For example, it is possible to supply power to an SMP, which is a system that can be used as a PSoC microcontroller from a single 1.5V battery.

Features of ARM DEVELOPMENT KIT Processor:

16-bit/32-bit ARM7TDMI-S microcontroller in a tiny LQFP64 package. 8 kB to 40 kB of on-chip static RAM and 32 kB to 512 kB of on-chip flash memory. 128-bit wide interface/accelerator enables high-speed 60 MHz operation. In-System/In-Application Programming (ISP/IAP) via on-chip boot loader software.

D Single flash sector/full chip erase in 400 ms and programming of 256 bytes in 1 ms. USB 2.0 Full-speed compliant device controller with 2 kB of endpoint RAM. The LPC2146/48 provides 5 kB of on-chip RAM accessible to USB by DMA.

D One or two (LPC2141/42 vs. LPC2144/46/48) 10-bit ADCs provide a total of 6/14 analogue inputs, with conversion times as low as 2,44 HS per channel. Single 10-bit DAC provides variable analogue output (LPC2142/44/46/48 only). Two 32-bit timers/external event counters (with four capture and four compare channels each), PWM unit (six outputs) and watchdog.

O Low power Real-Time Clock (RTC) with independent power and 32 kHz clock input. Multiple serial interfaces including two UARTs (16C550), two Fast 12C-bus (400 kb/s), SPI and SSP with buffering and variable data length capabilities.

Vectorized Interrupt Controller (VIC) with configurable priorities and vector addresses. Up to 45 of 5 V tolerant fast general purpose I/O pins in a tiny LQFP64 package. Up to 21 external interrupt pins available

O 60 MHz maximum CPU clock available from programmable on-chip PLL with settling time of 100 µs. An on-chip integrated oscillator operates with an external crystal from 1 MHz to 25 MHz. Power saving modes include Idle and Power-down.

D Individual enable/disable of peripheral functions as well as peripheral clock scaling for additional power optimisation. Processor wake-up from Power-down mode via external interrupt or BOD. Single power supply chip with POR and BOD circuits: CPU operating voltage range of 3.0 V to 3.6 V (3.3 V $\pm$ 10 %) with 5 V tolerant I/O pads.

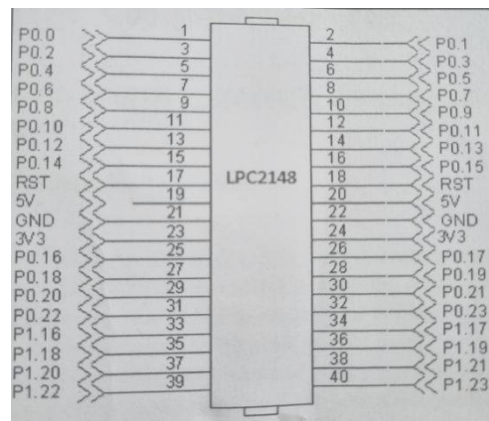
Power Supply:

- The external power can be AC or DC, with a voltage between 9V/12V, 1A output, at 230V AC input. The ARM board produces +5V using an LM7805 voltage regulator, which supplies the peripherals.
- LM1117 Fixed +3.3V positive regulator used for processor & processor-related peripherals.

Flash Programming Utility:

- NXP (Philips) - NXP Semiconductors produce a range of Microcontrollers that feature both on-chip flash memory and the ability to be reprogrammed using In-System Programming technology.

Pin Configuration:



On-board Peripherals:

- 8-Nos. of Point LED's (Digital Outputs)
- 8-Nos. of Digital Inputs (slide switch)
- 2 Lines X 16 Character LCD Display
- I2C Enabled 4 Digit Seven-Segment Display
- 128x64 Graphical LCD Display
- 4 X 4 Matrix keypad
- Stepper Motor Interface
- 2 Nos. Relay Interface
- Two UART or serial communication through the PC port
- Serial EEPROM
- On-chip Real Time Clock with battery backup
- PS/2 Keyboard interface(Optional,
- Temperature Sensor
- Traffic Light Module(Optional)
- Buzzer(Alarm Interface)

RESULT:

Thus, the study of the ARM processor has been done and ensured its composition with internal features.

FLASHING OF LEDS

Objective:

To verify the flashing of LEDS in ARM DEVELOPMENT KIT microcontroller board using embedded C program.

Software tools Requirement: Keil software

PROCEDURE:

Step 1: Open the µvision 4 icon on the desk top, it will generate a window as shown below

Step 2: To create new project go to project select new micro vision project.

Step 3: select a drive where you would like to create your project.

Step 4: Create a new folder and name it with your project name.

Step 5: Open that project folder and give a name of your project executable and save it.

Step 6: After saving it will show some window there you select your microcontroller company i.e NXP from Phillips.

Step 7: Select your chip as ARM DEVELOPMENT KIT

Step 8: After selecting chip click on OK then it will display some window asking to add STARTUP. Select YES.

Step 9: A target is created and startup is added to your project target and is shown below.

Step 10: To write your project code select a new from menu bar.

Step 11: It will display some text editor, to save that select SAVE option from menu bar.

Step 12: By giving a name with extension .C and save it

Step 13: Write the code of your project and save it

Step 14: To add our c to target give a right click on Source Group, choose "ADD s to Group" option.

Step 15: It will display some window there select the you have to add and click on ADD option.

Step 16: It will be added to our target and it shows in the project window.

Step 17: Right click on target in the project window and select "Options for Target"

Step 18: It will show some window, in that go to output option and choose Create Hex option by selecting that box.

Step 19: In the same window go to Linker option and choose Memory Layout from Target Dialog by selecting the box, and click OK.

Step 20: Compile your project go to Project select Build Target option or press F7.

Step 21: observe the flashing of LED's in the ARM board.

Prelab Questions:

1. What is the blinking of LEDs?

LED blinking refers to the process of continuously turning an LED (Light Emitting Diode) on and off in a repetitive pattern. It is a simple and common demonstration in electronics and microcontroller-based projects.

2. What is a flashing circuit?

Flashing circuits are very interesting. They grab your attention and can be used in many applications. They consume very little energy and some circuits use a single cell that can last as long as 12 months. You can flash globes, LEDs and pulse all types of devices such as motors, solenoids and piezo diaphragms.

Program:

```
#include <lpc214x.h>
unsigned int delay;
int main(void)
{
    IOODIR = (1<<10); // Configure P0.10 as Output
    while(1)
    {
        IOOCLR = (1<<10); // CLEAR (0) P0.10 to turn LED OFF
        for(delay=0; delay<500000; delay++); // delay
        IOOSET = (1<<10); // SET (1) P0.10 to turn LEDs ON
        for(delay=0; delay<500000; delay++); // delay
    }
}
```

Postlab Questions:

1. What is the purpose of blinking lights?

Flashing lights are designed to catch attention and are often used to convey warnings regardless of the color they are. These lights are commonly considered to be emergency responder lights.

Result:

Thus the Flashing of LEDs using ARM DEVELOPMENT KIT board was observed by using embedded C program successful.

Interfacing PWM with LPC 2148

Objective:

To develop and verify the interfacing of PWM with LPC 2148 ARM microcontroller

Software tools Requirement: Keil software

Procedure:

Step 1: Open the pvision 4 icon on the desk top, it will generate a window as shown below

Step 2: To create new project go to project select new micro vision project

Step 3: select a drive where you would like to create your project

Step 4: Create a new folder and name it with your project name

Step 5: Open that project folder and give a name x your project executable and save it

Step 6: After saving it will show some window there you select your microcontroller company i.e NXP from Phillips

Step 7: Select your chip as ARM DEVELOPMENT KIT

Step 8: After selecting chip click on OK then it will display some window asking to add STARTUP. Select YES.

Step 9: A target is created and startup is added to your project target and is shown below,

Step 10: To write your project code select a new from menu bar.

Step 11: It will display some text editor, to save that select SAVE option from menu bar

Step 12: By giving a name I with extension C and save it

Step 13: Write the code of your project and save it.

Step 14: To add our c to target give a right click on Source Group, choose "ADD s to Group" option.

Step 15: It will display some window there select the you have to add and click on ADD option.

Step 16: It will be added to our target and it shows in the project window.

Step 17: Right click on target in the project window and select "'Options for Target"

Step 18: It will show some window, in that go to output option and choose Create Hex option by selecting that box

Step 19: In the same window go to Linker option and choose Memory Layout from Target Dialog by selecting the box, and click OK.

Step 20: Compile your project go to Project select Build Target option or press F7.

Step 21: Check the concern block of output and observe the results

Prelab Questions:

1.What is PWM in LPC2148?

Pulse Width Modulation (PWM) is a technique by which width of a pulse is varied while keeping the frequency constant. A period of a pulse consists of an ON cycle (HIGH) and an OFF cycle (LOW) The fraction for which the Signal is ON over a period is known as duty cycle.
Duty Cycle (In %) = $\frac{\text{ON time}}{\text{Period}} \times 100$

2 What is PWM interface?

Description. The PWM Interface block simulates the PWM output of a hardware board. This block gets duty cycle data messages from a connected PWM Write block that can either generate a switching pulse-width-modulated waveform or pass the duty cycle value to the output

Program:

```
#include<PC214X.H>

void pwm_init (void)
{
    PINSELO 0x8002;
    PWMPCR = 0x600;
    PWMMRO = 150000;
    PWMTCR=0xx9;
    PWMMCR= x02;
}

int main (void)
{
    pwm initO; while(1)
    PWMMR1=75000;
    PWMMR2=90000;
    PWMLER 0x6;
}
```

Postlab Questions:

1. How to generate PWM from microcontroller?

The basic idea to generate PWM signal is using a counter or timer and a CMP (compare) value and a digital output pin. The counter continuously counts to up or down, and is compared with CMP value. The digital output (PWM) will be changed when the counter matches the CMP value or when counter resets.

2. What is the PWM principle?

Pulse width modulation or PWM is a commonly used control technique that signals from digital devices such as microcontrollers. In PWM technique, the signal is distributed through a series of pulses rather than a continuously varying (analog) signal.

RESULT:

Thus the interfacing of PWM with ARM processor has been verified and observed the output successfully.

Interfacing Keyboard and LCD with LPC 2148

Objective:

To develop and verify the interfacing of Keyboard and LCD with LPC 2148 ARM microcontroller.

Software tools Requirement: Keil software

Procedure:

Step 1: Open the uvision 4 icon on the desk top, it will generate a window as shown below

Step 2: To create new project go to project select new micro vision project.

Step 3: select a drive where you would like to create your project.

Step 4: Create a new folder and name it with your project name.

Step 6: After saving it will show some window there you select your microcontroller give a name of your project executable and save it company i.e NXP from Phillips

Step 7: Select your chip as ARM DEVELOPMENT KIT

Step 8: After selecting chip click on OK then it will display some window asking to add STARTUP. Select YES.

Step 9: A target is created and startup is added to your project target and is shown below.

Step 10: To write your project code select a new from menu bar.

Step 11: It will display some text editor, to save that select SA VE option from menu bar.

Step 12: By giving a name I with extension .C and save it

Step 13: Write the code of your project and save it.

Step 14: To add our c to target give a right click on Source Group, choose "ADD s to Group" option.

Step 15: It will display some window there select the you have to add and click on ADD option.

Step 16: It will be added to our target and it shows in the project window.)

Step 17: Right click on target in the project window and select "Options for Target"

Step 18: It will show some window, in that go to output option and choose Create Hex option by selecting that box

Step 19: In the same window go to Linker option and choose Memory Layout from Target Dialog by selecting the box. and click OK.,

Step 20: Compile your project go to Project select Build Target option or press F7.

Prelab Questions:

1.What is the interfacing of the LCD display?

ASCII code for the letters A to Z, a to 7, and numbers 0 to 9 is sent to the data lines (D0 -D7). These codes may be sent to LCD data lines through one port of 8255 (PPI). port A is used as the output port and send the data to the LCD. The EN pin and RS pin are connected to port B of the 8255.

2. How to interface keypad with arm?

The rows are connected to an output port and the columns are connected to an input port. To detect a pressed key, the microcontroller grounds all rows by providing 0 to the output latch, and then it reads the columns,

Program:

```
#include <LPC214x.H>

#define RS    0x00000400
#define CE    0x00001800

void clrscr(char ch);
void lcdinit(void);
void lcdcmd(char);
void lcddat(char);
void gotoxy(char,char);
void printstr(char *,char,char);
void wait (void);
void split_numbers(unsigned int number);

unsigned int thousands,hundreds,tens,ones;

void wait (void) {
    int d;
    for (d = 0; d < 100000; d++);
```

```

}

void lcdinit()
{

    IODIRO |= 0x00001CF0;
    IOCLR0 |= 0X00000FFF;
    lcdcmd(0x28);
    lcdcmd(0x06);
    lcdcmd(0x01);
    lcdcmd(0x0f);
    wait();
}

void gotoxy(char x, char y)
{
    if(y == 0)
        lcdcmd(0x80+x);
    else
        lcdcmd(0xc0+x);
}

void printstr(char *str, char x, char y)
{
    char i;
    gotoxy(x,y);
    wait();/(500);
    for(i=0;str[i]!='\0';i++)
        lcdat(str[i]);
}

void lcdcmd(char cmd)
{
    unsigned char LCDDAT;

```

```

    LCDDAT = (cmd & 0xf0);
    IOSET0 = LCDDAT;
    IOCLR0 = RS;
    IOSET0 = CE;
    wait();/(100);
    IOCLR0 = CE;
    IOCLR0 = 0X00000FFF;

    LCDDAT = ((cmd<<0x04) & 0xf0);
    IOSET0 = LCDDAT;
    IOCLR0 = RS;
    IOSET0 = CE;
    wait();/(100);
    IOCLR0 = CE;
    IOCLR0 = 0X00000FFF;
}

void lcddat(char cmd)
{
    unsigned char LCDDAT;
    LCDDAT = (cmd & 0xf0);
    IOSET0 = LCDDAT;
    IOSET0 = RS;
    IOSET0 = CE;
    wait();/(100);
    IOCLR0 = CE;
    IOCLR0 = 0X00000FFF;

    LCDDAT = ((cmd<<0x04) & 0xf0);
    IOSET0 = LCDDAT;
    IOSET0 = RS;

```

```

IOSET0 = CE;
wait();/(100);
IOCLR0 = CE;
IOCLR0 = 0X00000FFF;
}

```

```

void clrscr(char ch)
{
    if(ch==0)
    {
        printstr("      ",0,0);
        gotoxy(0,0);
    }
    else if(ch == 1)
    {
        printstr("      ",0,1);
        gotoxy(0,1);
    }
    else
    {
        lcdcmd(0x01);
    }
}

```

```

void split_numbers(unsigned int number)
{
    thousands = (number /1000);
    number %= 1000;
    hundreds = (number / 100);
    number %= 100;
    tens = (number / 10);
}

```



```

    number %= 10;
    ones = number ;
}

void Wait_Msg(void)
{
    lcdcmd(0x01);
    printstr(" PLEASE WAIT  ", 0, 0);
}

void Welcome_Msg(void)
{
    lcdcmd(0x01);
    printstr(" *ARM-7 LPC2148*", 0, 0);
    printstr("DEVELOPMENT KIT ", 0, 1);
}

int main (void)
{
    lcdinit();
    Wait_Msg();
    Welcome_Msg();
    while(1)
    {
    }
}

```

Postlab Questions:

1. How to interface LCD with PIC microcontroller?

We need to use LCD pins D4, D5, D6 and D7 with PIC16F877A pins D4, D5, D6, and D7 respectively. Also, connect power and ground to both the PIC and LCD. Connect power and ground to a potentiometer then wire the middle pin (variable part) to Vee pin of the LCD to be able to adjust the contrast

2. What interface is a keyboard?

Note 1: A keyboard interface allows users to keystroke to programs even if the native technology does not contain a keyboard. Example: A touchscreen PDA has a keyboard Interface built into its operating system as well as a connector for external keyboards.

Result:

Thus the interfacing of keyboard and LCD with ARM DEVELOPMENT KIT microcontroller using embedded C program has been verified successfully.

LED BLINK**Objective:**

The-objective of blinking an LED using an ARM7 microcontroller is to provide a hands-on introduction to embedded systems programming and microcontroller interfacing. laying the foundation for more complex projects and applications in the future.

Software tools Requirement: Keil software

Procedure:

Step 1: Open the uvision 4 icon on the desk top, it will generate a window as shown below

Step 2: To create new project go to project select new micro vision project.

Step 3: select a drive where you would like to create your project.

Step 4: Create a new folder and name it with your project name

Step 5: Open that project folder and give a name of your project executable and save it

Step 6: After saving it will show some window there you select your microcontroller company ie NXP from Phillips

Step 7: Select your chip as ARM DEVELOPMENT KIT

Step 8: After selecting chip click on OK then it will display some window asking to add STARTUP. Select YES.

Step 9: A target is created and startup is added to your project target and is shown below.

Step 10: To write your project code select a new from menu bar.

Step 11: It will display some text editor, to save that select SAVE option from- menu bar.

Step 12: By giving a name I with extension .C and save it

Step 13: Write the code of your project and save it.

Step 14: To add our c to target give a right click on Source Group, choose "ADD s to Group" option.

Step 15: It will display some window there select the you have to add and click on ADD option. U

Step 16: It will be added to our target and it shows in the project window.

Step 17: Right click on target in the project window and select "Options for Target"

Step 18: It will show some window, in that go to output option and choose Create Hex option by selecting that box

Step 19: In the same window go to Linker option and choose Memory Layout from Target Dialog by selecting the box, and click OK.

Step 20: Compile your project go to Project select Build Target option or press F7.

Prelab Questions:

1.How to blink LED using 8051?

Led Anode is connected to an external +5v series with 510 ohm resistor Resistor is used to limit the amount of current led is consuming. Led cathode is connected to 89c51 microcontroller pin directly. So when the microcontroller sources some voltage the led turs off' and when it grounds the led turns on.

2.How fast can LED blink?

Fast flashing LEDs have a flash rate of 2.5 flashes per second. Modern Police, Fire, Ambulance, and all other emergency vehicles use fast flashing lights and they are a great choice for all emergency situations and scenes.

Program:

```
#include <LPC214x.H>          /* LPC214x definitions */
void wait (void)              /* wait function */
{
    int d;
    for (d = 0; d <1000000; d++);    /* only to delay for LED flashes */
}
int main (void)
{
    IODIR0 = 0x80002000;          /* Declare P0.13 and P0.31 defined as Outputs */
    while (1)                    /* Loop forever */
    {
        IOCLR0 = 0x80002000;      /*Active Low outputs makes the LEDs ON port P0.13 and
P0.31*/
        wait ();
    }
}
```

```
IOSET0 = 0x80002000;    /* High outputs makes the LEDs OFF ON port P0.13 and  
P0.31*/  
    wait ();  
}  
}
```

Postlab Questions:

Why do LED lights blink?

The most common issue with flickering is the dimmer and downlight compatibility issues. The dimmer and downlights must talk together in sync. Using old dimmers designed for halogen lights on LED downlights will most likely cause flickering,

How do you count LED blinking?

If it's blinking slowly enough, you can count the flashes and time them with a stop watch. If it's blinking faster, you can connect an oscilloscope to the LED lead and measure the period. Some 'scopes can calculate the frequency.

Result:

Thus the implementation of LED BLINK protocol with ARM DEVELOPMENT KIT processor board using embedded C program has been done

STEPPER MOTOR

Objective:

Using an ARM7 microcontroller typically involves designing a system that can accurately control the rotation of the stepper motor

Software tools Requirement: Keil software

Procedure:

Step 1: Open the uVision 4 icon on the desktop to launch the software.

Step 2: Navigate to the Project menu and select "New MicroVision Project" to create a new project.

Step 3: Choose a drive where you want to create your project.

Step 4: Create a new folder with your project name in the selected drive.

Step 5: Open the newly created project folder and save your project with a name

Step 6: Select NXP from Philips as your microcontroller company and ARM Development Kit as your chip.

Step 7: Add startup code when prompted during the project setup.

Step 8: Create a new source file for your project.,

Step 9: Write your project code in the source file and save it.

Step 10: Add the source file to the project by right-clicking on Source Group and selecting "Add Files to Group."

Step 11: Right-click on the target in the project window and select "Options for Target."

Step 12: Choose "Create Hex" in the Output options

Step 13: Configure Memory Layout in the Linker options.

Step 14: Click OK to save the options

Step 15: Compile your project using the "Build Target" option or by pressing F7.

Prelab Questions:

I.How to control a stepper motor with a microcontroller?

Step & direction drivers simplify control of a stepper because they only require two signals from a microcontroller: take the direction pin high-or low to turn the motor's direction one way or the other. Ther pulse the step pin. With each pulse, the motor should step in the direction set by the direction pin

2.What is the working principle of stepper motor?

The basic working principle of the stepper motor is the following: By energizing one or more of the static phases, a magnetic field is generated by the current flowing in the coil and the rotor aligns with this field

Program:

```
#include <LPC214x.H>    /* LPC214x definitions */

#define step1    0x00010000 /* P1.16 */
#define step2    0x00020000 /* P1.17 */

void wait (void)
{
    /* wait function */
    int d;
    for (d = 0; d < 10000; d++);    /* only to delay for LED flashes */
}

call_stepper_forw()
{
    IOCLR1 = 0X00FF0000;
    IOSET1 = 0X00050000;
    wait();
    wait();
    wait();
    wait();
    IOCLR1 = 0X00FF0000;
    IOSET1 = 0X00070000;
    wait();
    wait();
    wait();
    wait();
    IOCLR1 = 0X00FF0000;
    IOSET1 = 0X00060000;
    wait();
}
```

```

wait();
wait();
wait();
IOCLR1 = 0X00FF0000;
IOSET1 = 0X00040000;
wait();
wait();
wait();
wait();
}

int main (void)
{
    IODIR1 |= 0xFFFFFFFF;
    IOCLR1 |= 0X00FF0000;
    wait();
    while(1)    /*Loop Forever*/
    {
        call_stepper_forw();
        wait();
        wait();
        wait();
        wait();
        IOCLR1 = 0X00FF0000;
    }
}

```


Postlab Questions:

1. How is a stepper motor used?

Commercially, stepper motors are used in floppy disk drives, flatbed scanners, computer printers, plotters slot machines, image scanners, compact disc drives, intelligent lighting, camera lenses, CNC machines, and 3D printers

2. What are the methods to control stepper motor?

The speed of the rotors rotation in a stepper motor may be controlled in a variety of ways. Three such possibilities are through a series resistance, gearbox and voltage regulation. Each type has advantages over each other, and a methods implementation depends on the motor application and functionality.

Result:

Thus the implementation of STEPPER MOTOR protocol with ARM DEVELOPMENT KIT processor board using embedded C program has

TEMPERATURE**Objective:**

Using a temperature sensor with an ARM7 microcontroller typically involves creating a system that can measure ambient temperature and potentially take action based on the temperature readings.

Software tools Requirement: Keil software

Procedure:

Step 1: Open the uVision 4 icon on the desktop to launch the software.

Step 2: Navigate to the Project menu and select "New MicroVision Project" to create a new project.

Step 3: Choose a drive where you want to create your project.

Step 4: Create a new folder with your project name in the selected drive.

Step 5: Open the newly created project folder and save your project with a name-

Step 6: Select NXP from Philips as your microcontroller company and ARM Development Kit as your chip.

Step 7: Add startup code when prompted during the project setup.

Step 8: Create a new source file for your project.

Step 9: Write your project code in the source file and save it.

Step 10: Add the source file to the project by right-clicking on Source Group and selecting "Add Files to Group."

Step 11: Right-click on the target in the project window and select "Options for Target."

Step 12: Choose "Create Hex" in the Output options

Step 13: Configure Memory Layout in the Linker options.

Step 14: Click OK to save the options.

Step 15: Compile your project using the "Build Target" option or by pressing F7.

Prelab Questions:

1. What is the processor used by ARM7?

ARM7 is a group of 32-bit RISC ARM processor cores licensed by ARM Holdings for microcontroller use. The ARM7 core family consists of ARM700, ARM710, ARM7DI, ARM710a, ARM720T, ARM740T, ARM710T, ARM7TDMI, ARM7TDMI-S, ARM7EJ-S. The ARM7TDMI and ARM7TDMI-S were the most popular cores of the family.

2.What are the features of ARM78 It has an on-chip static RAM of 8 kB-40 kB and an on-chip flash memory of 32 kB-512 kB. The wide interface is 128 bit, or accelerator allows 60 MHz high-speed operation.

Program:

```
#include <LPC214x.H>

#include "lcd.c"


void ADCInit (void)
{
    PINSEL1 |= 0x04000000;
    AD0CR  |= 0x00200204;
}

void ADC_StartConversion(void)
{
    AD0CR |= (1<<24);//Bits24=1 start ADC
}

void ADC_StopConversion(void)
{
    AD0CR &= (~(1<<24));//Bits24=0 stop ADC
}

unsigned int ADC_ReadChannel(void)
{
    unsigned long ADC_Val, t;
    ADC_StartConversion();
    while((AD0DR2&0x80000000)==0);
    if(AD0STAT & 0x00000400)
    {
        return(0);
    }
    t = AD0DR2;
```

```

ADC_Val = ((t>>6) & 0x000003FF);
ADC_StopConversion();
return(ADC_Val);
}

```

```

int main (void)

```

```

{

unsigned int adc_val;
unsigned int temp;
ADCInit();
lcdinit();
clrscr(10);
printstr("LM35",0,0);
wait();
wait();
wait();
clrscr(10);
printstr("TEMPERATURE",0,0);
wait();
while (1) /* Loop forever */
{
adc_val = ADC_ReadChannel();
temp = (unsigned int)((3.3*adc_val*100)/1024);
split_numbers(temp);
gotoxy(0,1);
lccdat(tens+0x30);
lccdat(ones+0x30);
gotoxy(2,1);

```

```
lccdat(0xDF);  
printstr("C",3,6) ;  
}  
}
```

Postlab Questions:

1.How can microcontroller be used to measure temperature?

The operation of the temperature measurement system is very simple Analog temperature is sensed by the temperature sensor. This temperature is then converted into digital format by the A/D converter. The microcontroller receives this data and displays the temperature on 3 digital displays

2. What is the use of AKM7 LPC2148"N

16-bit or 32-bit ARM7 Family-Based Microcontroller: The LPC2148 is based on the ARM7 architecture, offering developers the choice between 16-bit_and 32-bit processing power. This versatility allows the microcontroller to cater to a wide range of application requirements

Result:

Thus the implementation of Temperature protocol with ARM DEVELOPMENT KIT processor board using embedded C program has been done.

MATRIX KEYBOARD**Objective:**

Using a matrix keypad with an ARM7 microcontroller typically involves creating a system that can detect keypresses from a matrix arrangement of keys.

Software tools Requirement: Keil software

Procedure:

Step 1: Open the aVision 4 icon on the desktop to launch the software

Step 2: Navigate to the Project menu and select "New MicroVision Project" to create a new project.

Step 3: Choose a drive where you want to create your project.

Step 4: Create a new folder with your project name in the selected drive.

Step 5: Open the newly created project folder and save your project with a name

Step 6: Select NXP from Philips as your microcontroller company and ARM Development Kit as your chip.

Step 7: Add startup code when prompted during the project setup.

Step 8: Create a new source file for your project.

Step 9: Write your project code in the source file and save it.

Step 10: Add the source file to the project by right-clicking on Source Group and selecting "Add Files to Group."

Step 11: Right-click on the target in the project window and select "Options for Target."

Step 12: Choose "Create Hex" in the Output options.

Step 13: Configure Memory Layout in the Linker options.

Step 14: Click OK to save the options.

Step 15: Compile your project using the "Build Target" option or by pressing F7.

Prelab Questions:

1.How do you interface a matrix keyboard?

To directly interface a $R \times C$ matrix keypad to microcontroller you will need to allocate $R + C$ digital I/O pins. For example, the key press detection scheme illustrated in figure 3, requires a total of 7 I/O pins - four input pins to connect to the rows and three output pins to connect to the columns,

2.How to interface keypad with arm? The rows are connected to an output port and the columns are connected to an input port. To detect a pressed key, the microcontroller grounds all rows by providing 0 to the output latch, and then it reads the columns

Program:

```
#include <LPC214x.h>
```

```
#define TEMT 0x40
```

```
#define r1 (1<<16)
```

```
#define r2 (1<<17)
```

```
#define r3 (1<<18)
```

```
#define r4 (1<<19)
```

```
#define c1 (1<<20)
```

```
#define c2 (1<<21)
```

```
#define c3 (1<<22)
```

```
#define c4 (1<<23)
```

```
#define S7SEG_ENB 0x00B80000;
```

```
#define DIGI1_ENB 0x00800000;
```

```
#define S7SEG_LED 0xff000000;
```

```
unsigned int thousands,hundreds,tens,ones;
```

```
void init_Matrix_7seg(void)
```

```
{
```

```
    IODIR1 |= 0xffff0000; // set 7seg LEDs as output ports and matrix's MSB (COLUMN)as  
inputs and LSB (ROW)as outputs
```

```
    IODIR0 |= S7SEG_ENB; // set P0.19 p0.20 p0.22 P0.23 as outputs to drive 7seg enable  
pins
```

```
    IOPIN0 |= S7SEG_ENB; // since we are using common cathode 7 seg display, the enable  
signals
```

```

        // should be initially set to HIGH.

    }

void keypad_delay(void)
{
    unsigned int t1;
    t1=5000;
    while(t1--);
}

unsigned int keypad (void)
{
    unsigned int key;
    IOCLR1|=(c1|c2|c3|c4|r1|r2|r3|r4);
    while(1)
    {
        IOCLR1|=c1;
        IOSET1|=(c2|c3|c4);           // first column = 0

        if((IOPIN1&r1)==0)
        {
            key=1;
            keypad_delay();
            return key;
        }
        else if((IOPIN1&r2)==0)
        {
            key=5;

```



```

        keypad_delay();
        return key;
    }
else if((IOPIN1&r3)==0)
{
    key=9;
    keypad_delay();
    return key;
}
else if((IOPIN1&r4)==0)
{
    key=13;
    keypad_delay();
    return key;
}

IOCLR1|=c2;
IOSET1|=(c1|c3|c4);           //second column = 0

if((IOPIN1&r1)==0)
{
    key=2;
    keypad_delay();
    return key;
}
else if((IOPIN1&r2)==0)
{
    key=6;
    keypad_delay();

```

```

        return key;
    }
else if((IOPIN1&r3)==0)
{
    key=10;
    keypad_delay();
    return key;
}
else if((IOPIN1&r4)==0)
{
    key=14;
    keypad_delay();
    return key;
}

IOCLR1|=c3;
IOSET1|=(c1|c2|c4);           //third column = 0

if((IOPIN1&r1)==0)
{
    key=3;
    keypad_delay();
    return key;
}
else if((IOPIN1&r2)==0)
{
    key=7;
    keypad_delay();
    return key;
}

```

```

    }
else if((IOPIN1&r3)==0)
{
    key=11;
    keypad_delay();
    return key;
}
else if((IOPIN1&r4)==0)
{
    key=15;
    keypad_delay();
    return key;
}

IOCLR1|=c4;
IOSET1|=(c1|c2|c3);           //forth column = 0

if((IOPIN1&r1)==0)
{
    key=4;
    keypad_delay();
    return key;
}
else if((IOPIN1&r2)==0)
{
    key=8;
    keypad_delay();
    return key;
}

```

```

else if((IOPIN1&r3)==0)
{
    key=12;
    keypad_delay();
    return key;
}
else if((IOPIN1&r4)==0)
{
    key=16;
    keypad_delay();
    return key;
}
}
}

```

void Alpha_Display(unsigned int value)

```

{
    IOPIN0 |= S7SEG_ENB; // clear enables first
    IOPIN0 = ~DIGI1_ENB; // now enable only the digit1

    IOPIN1 &= ~S7SEG_LED; // make all the 7seg LED pins LOW
    switch(value)
    {
        /*a,b,c,d,e,f,g,h segment on off section
        0xff000000=>1111 1111 0000 0000 0000 0000 0000 0000
        =>hgfe dcba 0000 0000 0000 0000 0000 0000 for student reference
        */

        case 1: IOPIN1 |= 0x3F000000; break; //a,b,c,d,e,f ->ON, g,h->OFF for ZERO
        case 2: IOPIN1 |= 0x06000000; break; //b,c->ON, a,d,e,f,g,h->OFF for ONE
        case 3: IOPIN1 |= 0x5B000000; break; //a,b,d,e,g->ON, c,f->OFF for TWO
    }
}

```

```

case 4: IOPIN1 |= 0x4F000000; break; //a,b,c,d,g->ON    e,f->OFF for THREE
case 5: IOPIN1 |= 0x66000000; break; //b,c,f,g->ON    a,d,e,h->OFF for FOUR
case 6: IOPIN1 |= 0x6D000000; break; //a,b,c,f,g,h->ON    d,e->OFF for FIVE
case 7: IOPIN1 |= 0x7D000000; break; //a,c,d,e,f,g,h->ON    b->OFF for SIX
case 8: IOPIN1 |= 0x07000000; break; //a,b,c->ON    d,e,f,g->OFF for SEVEN
case 9: IOPIN1 |= 0x7F000000; break; //a,b,c,d,e,f,g,h->ON    for EIGHT
case 10: IOPIN1 |= 0x6F000000; break; //a,b,c,d,f,g,h->ON    e->OFF for NINE
case 11: IOPIN1 |= 0x77000000; break; //a,b,c,e,f,g,h->ON    d->OFF for A
case 12: IOPIN1 |= 0x7C000000; break; //c,d,f,g,h->ON    a,b->OFF for b
case 13: IOPIN1 |= 0x39000000; break; //a,d,e,f->ON    b,c,g->OFF for c
case 14: IOPIN1 |= 0x5E000000; break; //b,c,d,,e,g->ON    a,f->OFF for d
case 15: IOPIN1 |= 0x79000000; break; //a,d,e,f,g,h->ON    b,c->OFF for e
case 16: IOPIN1 |= 0x71000000; break; // a,f,g,h->ON    c,d,e->OFF for f
}
}

```

```

int main()
{
    unsigned int get_key, last_key, Disp_key;
    init_Matrix_7seg(); // Initialize matrix keyboard and 7segment display

    IOPIN1 &= ~S7SEG_LED; // make all the 7seg led pins to LOW
    IOPIN0 |= S7SEG_ENB // Disable all the 7 seg display

    last_key = 0; // Initialize this variable to zero

    while(1)
    {
        get_key = keypad(); // scan for a valid key press
        if(get_key != 0) // zero means no key is pressed

```

```

{
    if(get_key!= last_key) // check whether the same key is pressed again(assume this as
STEP1)

    {
        Disp_key = get_key; // valid new key is stored in another variable
        last_key = get_key; // this variable's value is used for STEP1
    }
}

Alpha_Dispay(Disp_key); /*this function is used to display number in hex format
(single digit only)*/

}
}

```

Postlab Questions:

1. What is a matrix keypad?

A matrix keypad is the kind of keypad you see on microwave ovens, gas pumps, and calculators. A matrix keypad you can connect to a breadboard is also great for prototypes and inventions where things like codes, times, or other values have to be entered

2 What is a matrix keyboard in microprocessor?

A keyboard matrix circuit is a design used in most electronic musical keyboards and computer keyboards in which the key switches are connected by a grid of wires, to a diode matrix. For example, 16 wires arranged in 8 rows and 8 columns can connect 64 keys for a full five octaves of range

Result:

Thus the implementation of MATRIX KEYBOARD with ARM DEVELOPMENT KIT processor board using embedded C program has been done